

PID2 represented-coordinate assurance

Exact binary64 reduction, guard boundaries, and evidence limits

REPRESENTED INPUTS · EXACT DYADICS · EXPLICIT NONCLAIMS

A self-contained account of the two-source reconstruction algebra, the selected binary64 arithmetic contract, exact witnesses and counterexamples, conditioning outcomes, checker architecture, resource bounds, negative results, and remaining proof obligations.

The integer-and-Fraction route is separately implemented: it differs from production in language, representation, and algorithm, but it is not independent estimation, external replication, or independent review.

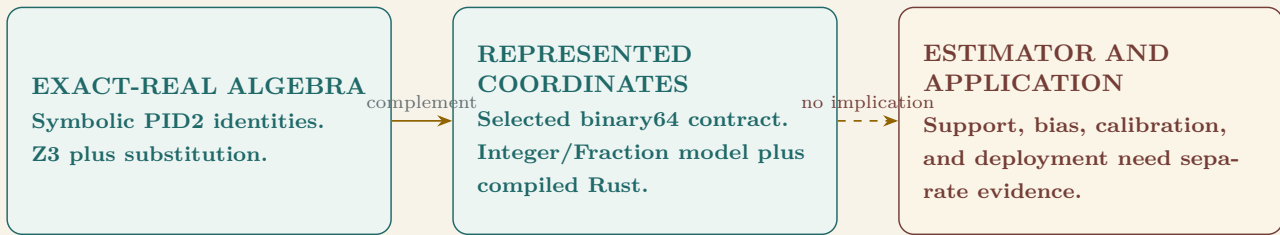
Sepehr Mahmoudian

31 August 2026

Canonical source: PID2_REPRESENTED_COORDINATE_ASSURANCE.md

Contents

Decision in one page	2
1. Provenance and novelty boundary	3
2. Assumptions stated before the equations	4
2.1 Exact binary64 decoding	4
2.2 Ordered-binary64 distance	4
3. Exact-real PID2 algebra and represented implementation	5
4. Why exact reduction is selected	6
5. Worked constructor witnesses	7
5.1 Finite identity erasure in both source directions	7
5.2 The 15/49 contract discriminator	7
5.3 Exact candidate versus Neumaier candidate	8
5.4 Exact reconstruction versus left association: 5/155	8
5.5 Exact reconstruction versus Neumaier overflow	9
5.6 All 16 signed-zero tuples	9
5.7 Near-zero 32/33 boundaries	10
5.8 Ordinary-scale 32/33 boundaries	10
6. Complete k=1..1023 acceptance-to-rejection family	11
6.1 Assumptions and parameter domain	11
6.2 Exact derivation of every member	11
6.3 Exact scaling reaches one rejected endpoint	12
6.4 What this family proves and does not prove	12
7. Overflow and false-bound controls	13
7.1 Finite atoms bounded by the joint magnitude can still reconstruct to infinity	13
7.2 Exact candidate synergy overflow	13
7.3 Sequential intermediate overflow with a finite exact answer	13
8. Conditioning outcomes	14
8.1 Assumptions before the diagnostic equations	14
9. Checker architecture and anti-cheating controls	15
10. Expected Rust and Python behavior	15
10.1 Rust constructor	15
10.2 Python binding parity obligation	15
11. Exact-real Z3 boundary	16
12. Cost and deployment interpretation	16
13. Negative results, supersession, and open work	16
13.1 Preserved negative result	16
13.2 Narrow supersession edge	16
13.3 Still open or deliberately unclaimed	16
14. Thirty-four-lens adversarial review	17
15. Reproduction and acceptance	18



Decision in one page

The continuous two-source PID constructor receives four **already computed binary64 estimator coordinates**:

- I1: the represented estimate of $I(S1; T)$;
- I2: the represented estimate of $I(S2; T)$;
- J: the represented estimate of $I((S1, S2); T)$; and
- R: the represented shared-exclusions redundancy estimate.

The selected project-defined arithmetic policy is:

1. preserve R exactly as supplied;
2. compute each unique atom with one binary64 subtraction;
3. compute synergy by summing the exact dyadic values of the four represented inputs, then round once to binary64 with ties to even;
4. reject a non-finite input or atom; and
5. reconstruct I1, I2, and J with the exact represented-input reducer and reject when any reconstructed coordinate is more than 32 positions away in the declared ordered-binary64 map.

This policy gives one source-order-independent meaning to the represented synergy formula. It also refuses to change synergy merely to compensate for rounding in the two unique atoms. The inclusive 32-position guard is a compatibility policy. It is not a proof that an accepted tuple is accurate: an accepted near-zero case can lose 100% of a coordinate in relative terms.

The new revision-4 evidence is deliberately narrower than an estimator theorem. A separately implemented integer-and-Fraction checker derives binary64 expectations without performing host floating-point arithmetic. It covers both finite identity-erasure directions, the 15/49 and 5/155 discriminators, two exact-versus-Neumaier discriminators, all 16 signed-zero tuples, the positive and negative near-zero boundaries, ordinary 32/33 boundaries, a complete 1-through-1023 scale family, overflow controls, and all five conditioning outcomes. Source custody and compiled Rust execution are separate scopes. The checker differs from production in language, numeric representation, and algorithm, and it never calls the Rust reducer. It still shares the human IEEE 754 specification, the same represented report inputs, repository custody, and institutional context. It is therefore not independent estimation, external replication, or independent review. A hostile self-test forbids host-float oracle shortcuts, runs from a copied root, and requires separately applied semantic and custody mutations to fail closed.

The evidence supports a **represented-coordinate engineering claim**. It does not show that a probability distribution or the Ehrlich/KSG estimator can attain any synthetic witness; validate a support declaration; prove estimator consistency, calibration, or uncertainty coverage; prove that Rust refines the exact model on every possible input; identify an error in Ehrlich et al.; or transfer the result to Williams--Beer, BROJA, categorical shared exclusions, or another PID.

1. Provenance and novelty boundary

The four-coordinate real algebra is the usual two-source PID reconstruction used with the continuous shared-exclusions construction of Ehrlich et al. (2024). The meanings of redundancy, unique information, and synergy come from that scientific context. This document does not redefine those objects.

Binary64 encoding, round-to-nearest with ties to even, signed zero, gradual underflow, and overflow behavior follow IEEE 754-2019. The compensated alternative called “Neumaier” refers to A. Neumaier, “Rundungsfehleranalyse einiger Verfahren zur Summation endlicher Summen,” *ZAMM* 54(1), 39--51 (1974), doi:10.1002/zamm.19740540106. Neumaier summation is a valuable comparison algorithm; it is not the selected PID2 reconstruction guard.

The following items are **project-defined engineering in pid-rs**, not claims of new PID mathematics:

- correctly rounded exact reduction of already represented finite binary64 operands;
- the inclusive 32-position ordered-binary64 compatibility guard;
- its typed rejection boundaries and conditioning report;
- the 1-through-1023 scale-family counterexample;
- the exact-versus-Neumaier and left-association discriminator witnesses;
- the separately implemented executable checker, hostile mutations, custody scopes, and API parity checks.

“Exact” below always modifies the sum of represented binary64 operands. It never means that an MI or redundancy estimator equals its population estimand.

2. Assumptions stated before the equations

All equations in Sections 3--12 use these assumptions unless a section states a stronger one.

1. I1, I2, J, and R are binary64 bit patterns, not exact population information quantities.
2. Inputs to the checked constructor must be finite. Negative represented coordinates are allowed; the constructor does not impose information-theoretic feasibility.
3. Basic binary64 operations use round-to-nearest with ties to even, gradual underflow, and one destination-format rounding. There is no excess-precision or double-rounding step in the model.
4. The exact reducer first interprets each finite binary64 input as its exact dyadic rational, adds those rationals, and rounds the total once to binary64. Exact cancellation is canonicalized to positive zero.
5. Every information coordinate uses the same unit, here nats, and the same estimator configuration. The constructor cannot verify this semantic precondition.
6. No probability law, sampling model, population support, estimator consistency, or attainability assumption is made for a raw-bit constructor witness.
7. When the full continuous estimator is used, its separate support, common-row, KSG/ISX configuration, and sampling assumptions still apply. Passing this arithmetic guard does not discharge them.
8. Multiplication by 2^k in the scale-family witness is an exact transformation of represented numbers. It is not a change of source variables, a probability-law transformation, a justified log-base conversion, or a gauge invariance theorem for PID.

2.1 Exact binary64 decoding

Let a finite binary64 bit pattern have sign bit s , exponent field e , and fraction field f . Under the assumptions above, its exact real value is

$$x(s, e, f) = \begin{cases} (-1)^s f 2^{-1074}, & e = 0, \\ (-1)^s (2^{52} + f) 2^{e-1075}, & 1 \leq e \leq 2046. \end{cases}$$

Thus every finite binary64 number lies on the integer grid with quantum 2^{-1074} . The checker implements this formula with integers and rational numbers. It never converts an expected value through a Python float, math, struct, NumPy, or decimal-floating route.

2.2 Ordered-binary64 distance

For a 64-bit pattern b , define

$$\phi(b) = \begin{cases} b \text{ OR } 2^{63}, & b \text{ has positive sign,} \\ \text{NOT } b \pmod{2^{64}}, & b \text{ has negative sign.} \end{cases}$$

The compatibility distance is $d(a, b) = |\phi(a) - \phi(b)|$. This gives adjacent finite binary64 values adjacent ordered integers, while preserving the distinction between negative and positive zero. It is a count of representable positions, not a number of nats and not a relative error.

3. Exact-real PID2 algebra and represented implementation

Under exact-real arithmetic, the two-source atoms satisfy

$$R = R, \quad U_1 = I_1 - R, \quad U_2 = I_2 - R,$$

and

$$S = J - I_1 - I_2 + R.$$

Substitution gives the three exact identities

$$R + U_1 = I_1, \quad R + U_2 = I_2,$$

and

$$R + U_1 + U_2 + S = J.$$

These are algebraic identities once the four definitions are assumed. They do not establish that the coordinates are compatible estimates of one population law. The Z3 obligations check this exact-real skeleton over linear real arithmetic; Z3 does not model binary64, Rust, KSG, or the shared-exclusions estimator.

For represented inputs, pid-rs deliberately uses different rounding contracts:

$$U_{1,64} = \text{RN}_{\text{even}}(I_{1,64} - R_{64}), \quad U_{2,64} = \text{RN}_{\text{even}}(I_{2,64} - R_{64}),$$

and

$$S_{64} = \text{RN}_{\text{even}}(\text{exact}(J_{64} - I_{1,64} - I_{2,64} + R_{64})).$$

Because the unique atoms are rounded separately, correctly rounding S64 does not force the rounded atom tuple to reconstruct J64. The constructor therefore performs three additional exact represented-input reductions and requires

$$d(\text{RN}_{\text{even}}(R_{64} + U_{1,64}), I_{1,64}) \leq 32,$$

$$d(\text{RN}_{\text{even}}(R_{64} + U_{2,64}), I_{2,64}) \leq 32,$$

and

$$d(\text{RN}_{\text{even}}(R_{64} + U_{1,64} + U_{2,64} + S_{64}), J_{64}) \leq 32.$$

Every displayed sum inside these three checks is exact over its represented operands before one final rounding. A non-finite reconstruction fails the check even if an ordered distance could otherwise be formed.

4. Why exact reduction is selected

At least twelve routes were considered.

Route	Strength	Decisive limitation or disposition
Ordinary left association	Cheapest and familiar	Source/order dependent; 5/155 witness lies outside the guard under this route.
Right association	Same operation count	Merely chooses a different incidental order.
Balanced pairwise tree	Reduces depth	Still tree dependent and not a single represented-input meaning.
Magnitude-sorted addition	Often accurate	Sorting and tie policy become another semantic dependency.
Kahan compensation	Useful error reduction	Not guaranteed to return the correctly rounded exact sum.
Neumaier compensation	Handles more magnitude patterns	One candidate differs by one bit; one atom reconstruction becomes NaN through overflow.
Long fixed-grid accumulator	Order independent and correctly rounded	Selected production route; fixed small cost for this four-term problem.
Python integer/Fraction model	Different language and representation; does not call the production reducer	Selected oracle; still shares the human IEEE specification, represented inputs, repository custody, and institutional context.
Exact-real Z3 algebra	Proves the symbolic identities	Selected complement; says nothing about binary64 refinement.
Exhaustive $k=1 \dots 1023$ family	Closes the declared exponent range	Selected bounded proof-by-enumeration plus symbolic derivation; not all input tuples.
Random or evolutionary search	Finds candidate counterexamples	Discovery aid only; seed/corpus searches cannot prove absence.
Arbitrary-precision decimal replay	Convenient high precision	Rejected as primary oracle because decimal precision and conversion add a second rounding policy.
IEEE bit-vector/QF_FP proof	Could cover bit-level obligations	Valuable future route; not claimed in this milestone.
Interval arithmetic	Can enclose results	Unnecessary for exact dyadic inputs and weaker than exact rational equality here.

The selected combination is not “formal tools alone.” Exact rational derivation supplies expected bits, compiled Rust exercises the production path, Python bindings test the public foreign-language surface, Z3 checks only the real algebra it can legitimately express, and hostile mutations test whether the checks detect nearby wrong claims.

5. Worked constructor witnesses

Every subsection restates any additional premise before using a formula. Hexadecimal values are the exact binary64 payloads. They are not decimal approximations copied from production output.

5.1 Finite identity erasure in both source directions

Additional premise: this is an arithmetic stress tuple, not a claimed estimator output. Let H be the finite payload 0x7e37e43c8800759c (the binary64 representation used by the Rust 1e300 literal). Test both

```
(I1,I2,J,R) = (1,0,0,H)
(I1,I2,J,R) = (0,1,0,H)
```

For either tuple, subtraction rounds both unique atoms to -H; the unit offset is too small to survive at the scale of H. The exact candidate synergy rounds to H. The atom bits are

```
R  0x7e37e43c8800759c
U1 0xfe37e43c8800759c
U2 0xfe37e43c8800759c
S  0x7e37e43c8800759c
```

The joint reconstruction is exact zero. One source reconstruction is also exact zero when its supplied coordinate is one, so its ordered distance is 4607182418800017408, far beyond 32. The other zero source reconstructs exactly. Swapping the unit coordinate swaps which source identity fails. Expected API result: `PidError::NumericalInstability` with the reconstructed-coordinate guard context in both directions.

Why needed: a joint-only check would admit a tuple that destroys one supplied source coordinate.

5.2 The 15/49 contract discriminator

Additional premise: all four inputs below are finite represented values.

```
I1 0x46d2fa52b582f1f7
I2 0xc6df8d0396df4fce
J  0x4688aebf760d4051
R  0x46df89c68c947ee9
```

Decode each payload by Section 2.1. One rounded subtraction per unique atom gives

```
U1 0xc6c91ee7ae2319e4
U2 0xc6ef8b6511b9e75c
```

The historical left-associated candidate and selected exact candidate are

```
historical S 0x46e670f6b4d0a362
exact S      0x46e670f6b4d0a361
```

Exact reconstruction of the two resulting atom tuples gives

```
with historical S 0x4688aebf760d4060 distance from J = 15
with exact S     0x4688aebf760d4020 distance from J = 49
```

The selected exact candidate therefore rejects, because 49 exceeds 32. This is intentional. The constructor gives the synergy formula one correctly rounded represented-input meaning; it does not choose a different synergy bit merely because that bit compensates better for separate rounding in U1 and U2.

Why needed: “closer reconstruction” and “correctly rounded candidate formula” are distinct objectives. This witness prevents an undocumented switch between them.

5.3 Exact candidate versus Neumaier candidate

Additional premise: Neumaier is evaluated in the declared term order $(J, -I1, -I2, R)$.

```
I1  0x43950f6578a693c8
I2  0xc373587698e10b0b
J   0x43996dc603da8752
R   0xbc469a3c16c4c8c1
```

The exact rational total rounds to $0x438268fc62d86c99$. The Neumaier route returns $0x438268fc62d86c9a$, one adjacent binary64 value higher. The exact-route constructor accepts and returns the first payload as synergy.

Why needed: compensated summation is not a synonym for correctly rounded exact summation. The test discriminates the algorithms instead of allowing them to agree accidentally on all fixtures.

5.4 Exact reconstruction versus left association: 5/155

Additional premise: the four atom fields are formed by the selected constructor before comparing reconstruction routes.

```
(I1, I2, J, R)
= (0x4072352e28889826,
   0x404488b4ebc88f45,
   0x3ff843725bb2f39b,
   0x404281666d288d41)
```

The returned atom bits are

```
R   0x404281666d288d41
U1  0x406fca02b5c70cfc
U2  0x40103a73f5001020
S   0xc0725dd48600e573
```

The exact atom sum rounds to $0x3ff843725bb2f3a0$, five ordered positions from J, so the tuple passes. Left association returns $0x3ff843725bb2f300$, 155 positions from J, and would reject. Neumaier returns the same payload as the exact route on this witness; that agreement is only a negative control for this tuple.

Why needed: replacing the exact guard with a left-associated guard changes the public admission set even when the atom definitions are unchanged.

5.5 Exact reconstruction versus Neumaier overflow

Additional premise: define $B=2^{1023}$. The raw inputs represent

$$I_1 = I_2 = \frac{3B}{4}, \quad J = \frac{3B}{2}, \quad R = -B.$$

Their payloads are

```
I1  0x7fd8000000000000
I2  0x7fd8000000000000
J   0x7fe8000000000000
R   0xffe0000000000000
```

The rounded unique atoms are each $7B/4$; the exact synergy is $-B$. Thus

```
(R,U1,U2,S)
= (0xffe0000000000000,
   0x7fec000000000000,
   0x7fec000000000000,
   0xffe0000000000000).
```

Exact arithmetic reconstructs

$$-B + \frac{7B}{4} + \frac{7B}{4} - B = \frac{3B}{2} = J.$$

The Neumaier state (running,correction) evolves as follows in this atom order:

Step	Term	Running	Correction	Reason
1	$-B$	$-B$	0	Exact first addition.
2	$7B/4$	$3B/4$	0	Exact cancellation.
3	$7B/4$	$+\infty$	$-\infty$	The rounded running sum overflows; compensation records the opposite infinity.
4	$-B$	$+\infty$	NaN	The correction path encounters an infinity subtraction.

Expected API result: the exact constructor accepts with zero reconstruction distance. A guard that used this Neumaier route would reject or become non-numeric. The Neumaier candidate synergy itself still equals $-B$; the discriminator is specifically the **atom-reconstruction guard**.

Why needed: the older record left exact-versus-Neumaier guard equivalence open. This explicit witness disproves global equivalence without alleging that Neumaier summation is generally flawed.

5.6 All 16 signed-zero tuples

Additional premise: each input independently takes $+0$ or -0 , giving $2^4=16$ tuples. IEEE subtraction yields -0 for $-0-(+0)$ and $+0$ for the other zero-minus-zero sign combinations used here. The exact reducer canonicalizes a zero total to $+0$.

For every tuple:

- output R preserves the input redundancy zero sign;
- U1 is -0 exactly when $I1=-0$ and $R=+0$, otherwise $+0$;
- U2 follows the analogous field-local rule;
- S is $+0$; and
- the tuple passes the inclusive guard.

Why needed: a single all-positive-zero test cannot detect sign propagation in either unique field, and a set-based enumeration could accidentally collapse $+0$ and -0 before testing all tuples.

5.7 Near-zero 32/33 boundaries

Additional premise: let $q=2^{-1074}$ and use

$$(I_1, I_2, J, R) = (pq, 1, pq, 1).$$

For $p=32$ or 33 , $RN(pq-1)=-1$, $U2=+0$, and the exact synergy is $+0$. Both the first source and joint reconstructions therefore become $+0$; their ordered distance from the positive subnormal pq is exactly p . Payload 32 passes and payload 33 fails.

For the negative counterpart $I1=J=-pq$, the ordered map crosses -0 and $+0$, so the distance from $-pq$ to reconstructed $+0$ is $p+1$. Negative payload 31 passes at distance 32; negative payload 32 fails at distance 33.

Why needed: this proves the threshold is inclusive and exposes its signed-zero asymmetry. It also shows why the threshold is not a relative-accuracy guarantee: the accepted coordinate can be reconstructed as zero.

5.8 Ordinary-scale 32/33 boundaries

Additional premise: unlike Section 5.7, these fixtures do not rely on subnormal-to-zero collapse.

Case	(I_1, I_2, J, R) payloads	Exact reconstructed J	Distance	API
32	c0bf8d662b97a649, 41093d1a13e4c232, c0735eb836c12cc0, 40cce37d33839b57	c0735eb836c12ca0	32	accept
33	bfe4ec1992f1b794, bfc4614c6039e98b, 3f76b2af711e5a91, bfe72423a04e5292	3f76b2af711e5a70	33	reject

The respective synergy payloads are c1067c266b6b2be1 and 3fb86d734ca0e0cf.

Why needed: near-zero cases alone could test only a special signed-zero edge. These fixtures pin the same inclusive decision at ordinary magnitudes.

6. Complete k=1..1023 acceptance-to-rejection family

6.1 Assumptions and parameter domain

This section additionally assumes an integer k with $1 \leq k \leq 1023$. All constructed nonzero values are positive normal binary64 numbers. The scale factor is the exactly represented power of two $F_k = 2^k$. No input product is subnormal or infinite.

Define the unscaled coordinates by raw fields:

```
I1(k) = +0
I2(k) exponent = 0x7fc-k, fraction = 2^52-2
J(k)  exponent = 0x7fe-k, fraction = 2^52-1
R(k)  = +0
```

The exponent ranges are:

- I2: 2043 down to 1021;
- J: 2045 down to 1023; and
- F_k : 1024 through 2046.

Every endpoint is therefore within the normal finite range. There are no omitted endpoint, subnormal, sign, or input-overflow exceptions in the declared 1..1023 family.

6.2 Exact derivation of every member

Set $q_k = 2^{(970-k)}$. Decoding the fields gives

$$I_2(k) = (2^{52} - 1)q_k$$

and

$$J(k) = (2^{54} - 2)q_k.$$

The exact synergy before rounding is

$$J(k) - I_2(k) = (3 \cdot 2^{52} - 1)q_k.$$

At that exponent, adjacent binary64 values are separated by $2q_k$. The exact value is halfway between $(3 \cdot 2^{52} - 2)q_k$ and $(3 \cdot 2^{52})q_k$. Ties-to-even selects the upper value,

$$S_{64}(k) = 3 \cdot 2^{52}q_k,$$

whose payload is

```
((0x7fe-k) << 52) | (1 << 51).
```

The exact represented atom total is

$$I_2(k) + S_{64}(k) = (2^{54} - 1)q_k.$$

This is halfway between $J(k) = (2^{54} - 2)q_k$ and the next power-of-two payload. Ties-to-even selects that next value, whose payload is $(0x7ff-k) << 52$. It is exactly one ordered position from $J(k)$. Therefore every unscaled member passes the 32-position guard.

6.3 Exact scaling reaches one rejected endpoint

Multiplication by $F_k=2^k$ shifts exponents exactly. Every one of the 1,023 members reaches the same represented endpoint:

```
I1'  0x0000000000000000
I2'  0x7cfffffffffffffe
J'   0x7fefffffffffffff
R'   0x0000000000000000
```

The endpoint synergy is finite:

```
S'   0x7fe8000000000000
```

However, the exact atom sum lies at the binary64 overflow midpoint and rounds to positive infinity. The constructor rejects through the reconstruction guard. The family therefore contains 1,023 distinct accepted seeds and 1,023 exact scaling cases that converge on **one** rejected endpoint; it does not contain 1,023 distinct rejected endpoint payloads.

For $k=4$, $F_4=2^4=16$, encoded as 0x4030000000000000. This is why the named example is the times-sixteen case. It is one member of the proved family, not an isolated search result.

6.4 What this family proves and does not prove

It proves that the project-defined admission policy is not homogeneous under these exact power-of-two transformations on the finite represented-coordinate domain. It does not prove a failure of the exact-real PID identities. It does not assert that multiplying all information coordinates by 2^k corresponds to a valid distribution, a change of logarithm base, a source rescaling, or an estimator gauge. It therefore cannot be called an Ehrlich or Wibrat defect.

7. Overflow and false-bound controls

7.1 Finite atoms bounded by the joint magnitude can still reconstruct to infinity

Additional premise: compare magnitudes as exact dyadic rationals, not by computing $16 \cdot J$ in binary64. Use

```
I1  0x7fe44dcfb32f7b9b
I2  0x7fbf796113915533
J   0x7fefffffffffffff
R   0x7fd1e3b3adc820e6
```

The derived atoms are

```
R   0x7fd1e3b3adc820e6
U1  0x7fd6b7ebb896d650
U2  0xffc40ab6d1c79732
S   0x7fe0b4de01426a31
```

Each atom is finite and each of $|U1|$, $|U2|$, and $|S|$ is at most $|J|$; this is stronger than the proposed exact-real premise $\max \leq 16|J|$. Nevertheless, their exact represented sum rounds to positive infinity. Expected API result: identity rejection.

Why needed: a magnitude premise on individual finite atoms does not guarantee that their sum is a finite binary64 result near the overflow boundary. This is a counterexample to that proposed sufficient bound, not to PID algebra.

7.2 Exact candidate synergy overflow

Additional premise: all four inputs are finite:

```
(I1, I2, J, R)
= (0xfc80000000000000,
   0xfc80000000000000,
   0x7fefffffffffffff,
   0xc800000000000000).
```

The exact represented candidate $J - I1 - I2 + R$ rounds to positive infinity. Expected API result: `PidError::NumericalInstability` with the atom context, before any identity comparison.

7.3 Sequential intermediate overflow with a finite exact answer

Additional premise: let M be maximum finite binary64 and use

$$(I_1, I_2, J, R) = (-M, M, M, 0).$$

The historical sequential first step $J - (-M)$ overflows. The exact represented linear form is

$$M - (-M) - M = M,$$

so selected synergy is finite M . The returned atoms are $(0, -M, M, M)$, all three identities pass, and the API accepts.

Why needed: rejecting on an incidental intermediate infinity would discard a tuple whose specified exact represented result is finite.

8. Conditioning outcomes

8.1 Assumptions before the diagnostic equations

The conditioning report describes cancellation in one already formed atom. It is not a condition number for the population estimator and not a standard error. For atom value a and its signed constituent terms t_i , define

$$A = \sum_i |t_i|,$$

then, when $A > 0$,

$$\rho = \frac{|a|}{A}.$$

When a is nonzero, the amplification diagnostic is

$$\kappa = \frac{A}{|a|} = \rho^{-1}.$$

Production uses its declared compensated sum for A . The exact checker uses fixtures whose absolute sums have unambiguous exact represented outcomes; it does not claim general equivalence between compensated and exact reduction.

Fixture (a; terms)	Exact represented outcome	Status	Why it is needed
(+0; +0,-0)	A=+0, no ratios	AllTermsZero	Separates absence of scale from cancellation.
(+0; +1,-1)	A=2, rho=+0, no kappa	ExactCancellation	Nonzero terms can cancel exactly.
(1/2; 1,-1/2)	A=3/2, rho=1/3, kappa=3	Finite	Pins both finite ratios and their expected bits.
(2 ⁻¹⁰⁷⁴ ; 1)	rho=2 ⁻¹⁰⁷⁴ ; kappa=2 ¹⁰⁷⁴ is not finite binary64	AmplificationExceedsF64Range	Distinguishes mathematical finiteness from binary64 representability.
(1; M,M)	A rounds to +inf	typed rejection	A non-finite scale cannot support the report.

The exact expected payload for rho=1/3 is 0x3fd5555555555555; kappa=3 is 0x4008000000000000.

9. Checker architecture and anti-cheating controls

The checker has three explicit scopes.

Scope	Exact model	Source custody	Compiled Rust
model	yes	explicitly not read	explicitly not run
model-source	yes	four exact regular-file digests and structural markers	explicitly not run
full	yes	same four files	focused debug and release test partitions

The exact model:

1. decodes each finite payload into an integer ratio;
2. performs addition, subtraction, multiplication, division, and ties-to-even encoding with integers and `fractions.Fraction`;
3. derives expected output bits without calling host binary64 arithmetic; and
4. separately emulates left-associated and Neumaier routes only as negative controls.

The hostile self-test:

- rejects any float literal, `float/fromhex` call, or import of `math`, `struct`, `decimal`, `NumPy`, or array-based numeric shortcuts in the oracle;
- copies the checker and all four pinned production/test files to a temporary root;
- runs both `model` and `model-source` there under normal and optimized Python with no live-root dependency;
- proves `model` does not read a supplied nonexistent source root;
- rejects changed, missing, or symlinked source files and a symlinked source root; and
- requires 18 separately applied semantic mutations to fail through the checker's declared error path, with no traceback and no altered success output.

Source hashes establish correspondence to reviewed same-repository bytes. They do not authenticate the repository, prove those bytes were compiled, or prove mathematical truth. The `full` scope adds compiled execution, but remains a bounded regression suite rather than exhaustive Rust refinement.

10. Expected Rust and Python behavior

10.1 Rust constructor

Expected typed outcomes are:

Condition	Outcome
Any non-finite input	<code>NumericalInstability</code> , input context
Any non-finite derived atom	<code>NumericalInstability</code> , atom context
Any non-finite or over-32-position reconstruction	<code>NumericalInstability</code> , reconstruction context
All checks pass	exact four atom payloads returned

The `exp0` diagnostic converts only `NumericalInstability` from this optional atom route into an explicit abstention. Any other `PidError` must propagate. This preserves error taxonomy; it does not turn `exp0` into an atom-estimator validation gate.

10.2 Python binding parity obligation

The Python scalar and report routes must expose the same four atom payloads as their one compiled Rust extension. The separately implemented expected route must reconstruct `U1`, `U2`, and `S` from the report's four represented estimate terms with integer-ratio arithmetic and the manual ties-to-even encoder. It must not compute expectations with `NumPy` or Python floating-point subtraction.

This parity check establishes one-build binding fidelity for the selected fixtures. It does not create an independent estimator, establish cross-platform bit identity, or expand the scientific validity of the Rust result.

11. Exact-real Z3 boundary

The SMT files prove five scoped exact-real obligations, including PID2 reconstruction, source-swap algebra, and the declared Möbius rows. The hostile self-test must separately mutate:

- a sign in the synergy row;
- a coordinate mapping under source exchange; and
- a Möbius reconstruction row.

Each wrong statement must become satisfiable and therefore fail the “unsatisfiable proof” gate. This is stronger than changing one shared offset in every file, because it tests distinct semantic dimensions. It remains exact linear-real reasoning. No SMT result is evidence about binary64 rounding, signed zero, overflow, estimator sampling, or production-code refinement.

12. Cost and deployment interpretation

The production reducer uses 34 fixed 64-bit limbs for positive magnitude and 34 for negative magnitude. One worst-case add is conservatively charged as 68 limb visits and one finalization as 136 limb visits. Additional premise: this is the declared conservative resource-envelope model, not a measured instruction or latency model. PID2 construction performs 12 represented addends across synergy plus the three identity reconstructions and four finalizations:

$$12 \cdot 68 + 4 \cdot 136 = 1360$$

conservative limb visits. These are resource-envelope units, not CPU instructions or latency measurements. The cost is constant with sample count once the four estimator coordinates exist; KSG and shared-exclusions estimation normally dominate the full call.

The exact Python assurance route is intentionally slower and runs in tests or release audit, not in the estimator hot path. The 1,023-family loop is bounded and operates on integers of roughly the binary64 exponent-grid width. It is not a proposed runtime estimator.

13. Negative results, supersession, and open work

13.1 Preserved negative result

The historical record at `audit/evidence/completion-run-ledger-2026-07-25.md` stated that exact-versus-Neumaier guard behavior remained open. That statement remains correct for the evidence available at that time and must not be rewritten.

13.2 Narrow supersession edge

Section 5.5 now supplies an explicit finite four-coordinate counterexample: exact atom reconstruction equals J, while the declared Neumaier trace ends in NaN. This supersedes only the old **global-equivalence uncertainty**. It does not supersede Neumaier's usefulness elsewhere, the older search record, or any estimator question.

13.3 Still open or deliberately unclaimed

- Exhaustive refinement of the Rust accumulator for every possible finite input tuple.
- A kernel-checked or QF_FP proof of the binary64 constructor and guard.
- Existence of probability laws or retained Ehrlich/KSG datasets that emit the synthetic stress tuples.
- Statistical calibration, estimator bias, joint atom uncertainty, or confidence coverage.
- Cross-platform/compiler bit identity beyond the tested build matrix.
- A theorem choosing 32 as an optimal error threshold. It remains compatibility policy.
- A general theorem relating the conditioning diagnostic to estimator error.
- Performance qualification on deployment hardware.

14. Thirty-four-lens adversarial review

Lens	Question	Revision-4 disposition
1	Semantic object	Says represented coordinates, never exact estimator outputs.
2	PID identity	Keeps Wibrat/Ehrlich, Williams--Beer, BROJA, and categorical SxPID scopes separate.
3	Exact algebra	Z3 and hand substitution cover only the real linear identities.
4	Encoding	Raw fields decode through the explicit dyadic formula.
5	Rounding	One ties-to-even encoder, with midpoint controls.
6	Signed zero	All 16 input tuples and field-local output signs enumerated.
7	Subnormal	Positive and negative near-zero thresholds derived separately.
8	Overflow	Candidate, reconstruction, and intermediate-overflow cases separated.
9	Scale	Complete $1 \dots 1023$ exact family; no gauge claim.
10	Guard topology	Both source identities and the joint identity checked.
11	Threshold	Ordinary and near-zero 32/33 controls pin inclusive behavior.
12	Conditioning	Every status/rejection branch has an exact fixture.
13	Source exchange	Exact candidate is order independent; no population symmetry inferred.
14	Alternative sums	Left, Neumaier, and exact routes have discriminating witnesses.
15	Error taxonomy	Input, atom, reconstruction, and unexpected errors remain distinct.
16	Public API	Direct constructor attack covers user-supplied estimates.
17	Python API	Scalar/report parity uses separately implemented integer-ratio expectations.
18	Debug/release	Focused compiled partitions run in both profiles.
19	Oracle separation	Python integer grid does not call the production reducer; specification, inputs, custody, and institutional context remain shared.
20	Oracle cheating	AST, copied-root, exact stdout, and mutations fail closed.
21	Source custody	Exact regular-file digests bind four reviewed paths.
22	Root isolation	Model/source replay succeeds from copied bytes without live root.
23	Hostile mutation	Rounding, sign, coordinate, row, threshold, and custody attacks included.
24	Resources	Fixed-limb envelope and 1,360 visits stated without latency inflation.
25	Performance	Assurance loop stays outside production; no benchmark promise.
26	Estimator	No attainability, consistency, or bias claim.
27	Statistics	No calibration, standard-error, or coverage claim.
28	Support/gauge	Estimator support and common-unit assumptions remain external obligations.
29	Provenance	Paper-defined algebra and project-defined arithmetic are labeled separately.
30	Publication	Worked assumptions, counterexamples, negative results, and commands are retained.
31	Downstream use	Binding fidelity is not authorization for a sensor or scientific decision.
32	Reproducibility	Scope-specific output states what was read and executed.
33	Formal limits	Z3 exact-real and compiled binary64 evidence are not conflated.
34	Historical integrity	Old inconclusive evidence is preserved with a narrow supersession edge.

15. Reproduction and acceptance

Run the exact-rational model and hostile self-test under both interpreter modes:

```
python3 -I -S -B scripts/check-pid2-represented-coordinate-v4.py --scope model
python3 -O -I -S -B scripts/check-pid2-represented-coordinate-v4.py --scope model
python3 -I -S -B scripts/check-pid2-represented-coordinate-v4-self-test.py
python3 -O -I -S -B scripts/check-pid2-represented-coordinate-v4-self-test.py
```

Add source custody without compiling:

```
python3 -I -S -B scripts/check-pid2-represented-coordinate-v4.py --scope model-source
python3 -O -I -S -B scripts/check-pid2-represented-coordinate-v4.py --scope model-source
```

Run model, source custody, and focused debug/release Rust tests:

```
python3 -I -S -B scripts/check-pid2-represented-coordinate-v4.py --scope full
python3 -O -I -S -B scripts/check-pid2-represented-coordinate-v4.py --scope full
```

The optimized invocation is not a different mathematical oracle. It detects accidental dependence on Python assertions or optimization-sensitive control flow. Acceptance additionally requires the Python binding parity tests, the separately applied Z3 mutations, method-catalog coherence, formatting, lint, broader workspace tests, source-state regeneration, a clean committed tree, and direct remote OID verification.

The publication PDF is a checked projection of this Markdown source:

```
scripts/build-pid2-represented-coordinate-assurance-pdf.sh
scripts/check-pid2-represented-coordinate-assurance-pdf.sh --exact
scripts/check-pid2-represented-coordinate-assurance-pdf.sh --cross-toolchain
```

The exact profile requires byte identity with a same-toolchain rebuild. The cross-toolchain profile permits binary serialization differences but requires the same extracted layout text, page geometry, mathematical sentinels, embedded-font contract, safe-link policy, and page-by- page rendering. Neither profile enlarges the mathematical, statistical, estimator, or external- replication claims made above.